

Guía Técnica del Código

Pokémon Battle Platform — Arquitectura Elixir/OTP

Explicación de Conceptos, Patrones y Decisiones de Diseño en el Proyecto

Universidad del Quindío

Programa de Ingeniería de Sistemas y Computación

Mayo 2026

1. Introducción

Este documento explica los conceptos clave de Elixir y OTP utilizados en la implementación de la plataforma. Cada sección incluye el contexto teórico, los archivos donde aparece el patrón, y ejemplos concretos extraídos del código del proyecto.

2. GenServer — Servidor de Proceso con Estado

Archivos: [gestor_entrenadores.ex](#), [batalla.ex](#), [gestor_salas.ex](#), [intercambio.ex](#), [servidor.ex](#)

Un GenServer (Generic Server) es un proceso OTP que mantiene estado en memoria y responde mensajes de forma controlada. Es el componente fundamental de la arquitectura concurrente del proyecto: cada batalla, sala de intercambio y gestor global vive en su propio proceso GenServer independiente.

Callback	Tipo de Mensaje	Uso
handle_call/3	Síncrono — el llamador espera respuesta	Consultas de perfil, inventario, iniciar sesión
handle_cast/2	Asíncrono — el llamador no espera	Agregar Pokémon, actualizar monedas
handle_info/2	Mensajes del sistema (timers, señales)	Timeouts de turno, desconexiones

```
# Llamada síncrona — el cliente ESPERA la respuesta
def handle_call({:perfil, usuario}, _from, estado) do
  {:reply, :ok, estado}
  # ↑ respuesta ↑ nuevo estado guardado
end

# Cast asíncrono — el cliente NO espera
def handle_cast({:agregar_pokemon, usuario, pokemon}, estado) do
  {:noreply, nuevo_estado}
end
```

3. DynamicSupervisor — Supervisión en Tiempo de Ejecución

Archivo: [supervisor_batallas.ex](#)

El DynamicSupervisor crea y supervisa procesos hijos en tiempo de ejecución. Con la estrategia `:one_for_one`, si una batalla falla, solo ese proceso se reinicia sin afectar las demás. Cada batalla vive en su propio proceso aislado.

```
def init(:ok),
```

```
do: DynamicSupervisor.init(strategy: :one_for_one)

def iniciar_batalla(config) do
  spec = {PokemonBattle.Batalla, config}
  DynamicSupervisor.start_child(__MODULE__, spec)
end
```

4. Árbol de Supervisión OTP

Archivo: `application.ex`

Define el orden de arranque y supervisión de todos los procesos del sistema. Los procesos listados en `children` se inician en secuencia y se supervisan automáticamente: si alguno muere inesperadamente, el supervisor lo reinicia.

```
def start(_type, _args) do
  children = [
    {PokemonBattle.SupervisorBatallas, []},
    {PokemonBattle.GestorEntrenadores, []},
    {PokemonBattle.GestorSalas, []},
    {PokemonBattle.Servidor, []}
  ]
  Supervisor.start_link(children, strategy: :one_for_one)
end
```

5. El Operador `|>` (Pipe)

Pasa el resultado de una expresión como primer argumento de la siguiente función. Permite escribir transformaciones de datos como una cadena de pasos legible de arriba hacia abajo, eliminando variables intermedias innecesarias.

```
# Ejemplo del ranking en gestor_entrenadores.ex:
ranking =
  estado
  |> Map.values() # lista de entrenadores
  |> Enum.sort_by(fn entrenador -> # ordenar por victorias
    {-entrenador.victorias, -entrenador.monedas_acumuladas}
  end)
```

6. Pattern Matching

Elixir compara la estructura de los datos, no solo su valor. El operador `^` (pin) fija el valor de una variable para comparar en lugar de reasignar.

```
case Map.get(estado, usuario) do
  nil ->
```

```
# El usuario no existe → registrar nueva cuenta
...
%{clave: ^clave} ->
  # Usuario existe Y clave coincide (^clave compara, no asigna)
  {:reply, {:ok, "Bienvenido"}, estado}
- ->
  {:reply, {:error, "Contraseña incorrecta"}, estado}
end
```

7. with — Encadenamiento de Pasos que Pueden Fallar

Ejecuta pasos en secuencia. Si alguno no coincide con el patrón esperado, salta directamente al bloque else, evitando anidaciones profundas de case y simplificando el manejo de errores.

```
# En transferir_pokemon (gestor_entrenadores.ex):
with %{} = entrenador_origen <- Map.get(estado, de),
      %{} = entrenador_destino <- Map.get(estado, hacia),
      pokemon when pokemon != nil <- Enum.find(entrenador_origen.inventario,
                                                &(&1.id == id)) do
  {:reply, {:ok, pokemon}, nuevo_estado}
else
  -> {:reply, {:error, "No se pudo transferir"}, estado}
end
```

8. Capture Operator & — Funciones Anónimas Cortas

Permite crear funciones anónimas de forma abreviada. El símbolo &1 representa el primer argumento de la función.

```
# Forma larga:
Enum.filter(inventario, fn pokemon -> pokemon.id == id end)

# Forma corta con capture operator:
Enum.filter(inventario, &(&1.id == id))

# Para referenciar funciones con nombre:
Enum.map(entrenadores, &stringify/1)
```

9. ETS — Tabla en Memoria Compartida

Archivo: intercambio.ex

ETS (Erlang Term Storage) es una tabla en memoria accesible por múltiples procesos simultáneamente. Se usa para registrar las salas de intercambio activas indexadas por código, permitiendo búsqueda rápida desde cualquier proceso del clúster.

```
:ets.new(:intercambios_activos, [:named_table, :public, :set])

# :named_table → accesible por nombre desde cualquier proceso
# :public      → cualquier proceso puede leer y escribir
# :set         → cada clave es única (como un mapa)

:ets.insert(:intercambios_activos, {codigo, self()})

case :ets.lookup(:intercambios_activos, codigo) do
  [{_codigo, pid}] -> pid    # encontrado
  []               -> nil    # no existe
end
```

10. Process.send_after — Timers Internos

Archivo: batalla.ex

Envía un mensaje al proceso después de N milisegundos. Es la base del sistema de timeout de turno: si un jugador no actúa en el tiempo límite, el proceso recibe automáticamente el mensaje `:timeout_turno` y asigna pasar como acción.

```
# Iniciar timer de turno:
ref = Process.send_after(self(), :timeout_turno, estado.tiempo_turno * 1 000)

# Cancelar si ambos actuaron antes del límite:
Process.cancel_timer(ref)

# El mensaje llega a handle_info cuando vence el tiempo:
def handle_info(:timeout_turno, estado) do
  # Se ejecuta automáticamente al expirar
  ...
end
```

11. Módulo de Persistencia con Jason

Archivo: persistencia.ex

La librería Jason serializa y deserializa JSON. Al leer datos, las claves son Strings por defecto; la función `atomizar/1` las convierte a átomos para poder usar notación de punto en el resto del código.

```
# Leer JSON → mapa Elixir (claves String):
{:ok, contenido} = File.read("data/trainers.json")
{:ok, data}      = Jason.decode(contenido)
# data = %{"usuario" => "Ana", ...}

# Mapa Elixir → escribir JSON:
contenido = Jason.encode!(data, pretty: true)
File.write!("data/trainers.json", contenido)
```

12. Fórmula de Daño — trunc vs round

El motor de combate usa trunc (truncado) en lugar de round (redondeo) para el cálculo de daño, siguiendo la especificación del proyecto. trunc elimina la parte decimal sin redondear, garantizando resultados deterministas.

```
trunc(13.9)  # → 13   (no redondea, elimina decimal)
round(13.9)  # → 14   (redondea al más cercano)

# Fórmula completa en motor_combate.ex:
dano_base = trunc((poder * (ataque / defensa)) / 5 + 2)
dano_final = trunc(dano_base * efectividad * stab * aleatorio)
```

13. Resumen de Archivos del Proyecto

Archivo	Función Principal
application.ex	Arranca el árbol OTP (punto de entrada de la aplicación)
servidor.ex	Lee comandos de consola y los enruta al módulo correcto
gestor_entrenadores.ex	Mantiene el estado global de todos los entrenadores en memoria
sistema_sobres.ex	Lógica de tienda, compra y apertura de sobres con rareza
motor_combate.ex	Fórmulas de daño, efectividad de tipos y cálculo STAB
batalla.ex	GenServer de una batalla activa (uno por batalla en curso)
supervisor_batallas.ex	Crea y supervisa los procesos de batalla e intercambio
gestor_salas.ex	Registro de salas de batalla disponibles
intercambio.ex	GenServer de sala de intercambio con timers y tabla ETS
persistencia.ex	Lectura y escritura de archivos JSON con Jason
cluster.ex	Selección de nodo para batallas distribuidas multi-nodo